Process Guide and Daily Routine

Conflict, Democracy and Arms. Nicolas and Yair. Data Analysis with Python, HS26. Presentation on Friday 4 December 2026, report due Thursday 10 December.

This guide explains how the two of you work on the project every day: where you write code, how it gets into the shared notebooks, and how you keep GitHub clean. The content of each task is in your sandbox notebook (`sandbox_nicolas.ipynb`, `sandbox_yair.ipynb`).

The short version

	Rule
1	Pull before you start. Every time.
2	Write and test in your own sandbox. Only you edit your sandbox.
3	A task is done when its checkpoint cell prints OK.
4	Paste only working, understood code into the main notebook.
5	Restart and Run All on the main notebook, then commit and push.
6	Tell your partner in one line what you pushed.
7	Write simple code you can explain line by line. More lines are fine.

1. One-time setup (both laptops)

Do this once together, ideally in the same room, so both laptops end up identical.

#	Step	Check
1	Install Python 3.11 or newer, VS Code with the Python and Jupyter extensions, Git and GitHub Desktop.	`python3 --version`
2	**Nicolas** creates the private repo `conflict-arms-democracy` (README, .gitignore template Python) and invites Yair and the tutors under Settings, Collaborators.	Yair accepts the email invite
3	**Nicolas** clones the repo to a simple path without spaces and outside iCloud or OneDrive, for example `~/projects/conflict-arms-democracy`.	Folder opens in VS Code
4	**Nicolas** copies the repo files from the zip into the repo folder (this creates all folders and files), puts the UCDP and SIPRI raw files in `data/raw/`, then commits and pushes. V-Dem stays local; it is over GitHub's 100 MB limit and already listed in `.gitignore`.	Files visible on GitHub
5	**Yair** clones the repo only after this push. Do not copy the zip a second time.	Yair sees all files
6	**Both:** create and activate a virtual environment in the repo folder and install the packages (commands below).	`import pandas` works
7	**Both:** run `nbstripout --install` inside the repo. Notebook outputs are then never committed, which prevents most merge conflicts.	Run once per laptop
8	**Both:** download the V-Dem file and put it in `data/raw/vdem/`.	`git status` does not list it
9	**Both:** in VS Code, select the `.venv` kernel for every notebook (top right).	Kernel shows .venv
10	**Both:** run `00_setup_check.ipynb`.	Last cell prints OK

```
cd ~/projects/conflict-arms-democracy
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
nbstripout --install
```

Files in the repo

```
conflict-arms-democracy/
  data/raw/            downloads, never changed
  data/raw/vdem/       V-Dem file, local only (in .gitignore)
  data/clean/          CSV files written by the notebooks
  notebooks/
    sandbox_nicolas.ipynb     only Nicolas edits
    sandbox_yair.ipynb        only Yair edits
    00_setup_check.ipynb ... 07_charts.ipynb   main notebooks
  app/
    streamlit_app.py
    panels_conflict.py (Nicolas)   panels_arms.py, panels_analysis.py (Yair)
    try_streamlit_nicolas.py   try_streamlit_yair.py   practice only
  docs/                build guide and this PDF
  AI_LOG.md   CONVENTIONS.md   requirements.txt   README.md
```

2. The daily routine

Follow these steps every time you sit down to work, even for 30 minutes. The order matters: pulling first and pushing last is what keeps your two copies in sync.

Start of a session

#	Step	GitHub Desktop	Terminal
1	Open the repo, activate the environment.	Open repo, Repository, Open in VS Code	`source .venv/bin/activate`
2	**Pull** the latest version from GitHub.	Fetch origin, then Pull origin	`git pull`
3	Read your partner's last message: what did they push, which clean files are new?	History tab	`git log --oneline -5`
4	Open **your sandbox** and run the setup cell.		

While you work

#	Step	GitHub Desktop	Terminal
5	**Write** your code in the empty cells of the current task. Try, print, look at the result. Open the hint only when stuck.		
6	Run the **checkpoint** cell. OK means the task is done. An error tells you what is still wrong.		
7	Optional but useful: **commit the sandbox** after each finished task, so you can go back.	Tick only your sandbox, write a message, Commit	`git add notebooks/sandbox_yair.ipynb` `git commit -m "..."`

Moving code into the main notebook

#	Step	GitHub Desktop	Terminal
8	Kernel, **Restart and Run All** on your sandbox. Everything must run from top to bottom.		
9	Open the main notebook for this task (for example `04_sipri.ipynb`) and **paste** only the cells that work. Leave out experiments and long printouts. Add a short markdown cell above each step.		
10	Restart and Run All on the **main notebook**. All checkpoints must print OK and the clean CSV must be written.		

End of a session

#	Step	GitHub Desktop	Terminal
11	**Commit** the main notebook and the new clean CSV with a clear message. Tick only your own files.	Tick the files, write summary, Commit to main	`git add notebooks/04_sipri.ipynb data/clean/sipri_cy.csv` `git commit -m "04: sipri_cy done"`
12	**Pull** in case your partner pushed meanwhile.	Fetch origin, Pull origin	`git pull`
13	**Push.**	Push origin	`git push`
14	**Tell your partner** in one line.		"Pushed 04, sipri_cy.csv is ready"

One owner per file. Only Nicolas edits `sandbox_nicolas.ipynb`, `02_ucdp`, `03_vdem` and `panels_conflict.py`. Only Yair edits `sandbox_yair.ipynb`, `04_sipri`, `panels_arms.py` and `panels_analysis.py`. Shared files (`01_crosswalk`, `05_merge`, `06_analysis`, `07_charts`, `streamlit_app.py`) are edited by one person at a time; say in your chat who has it.

How we write code

We are still learning pandas, and the grade depends on understanding every line. So we write code slowly and in small steps, even if it gets longer.

Do	Avoid
One operation per line, each result gets a name	Long chains like `df.groupby().agg().reset_index().rename()`
Print `shape` or `head()` after every step that changes the data	Running ten steps and only looking at the end
Loops, `if`, dictionaries and lists when they make the logic clear	Compact commands you found online and cannot explain
Try a new function on a tiny example first (see the merge practice in the sandbox)	Using a new function directly on 60,000 rows
Check one case by hand (for example India 2000) and compare	Trusting a result because it runs without an error
One comment above each block, in your own words	No comments, or comments copied from somewhere

Good commit messages

Good	Not useful
`03: regime spans and vdem.csv`	`update`
`04: drop non-state recipients`	`changes`
`05: merge done, panel.csv 1950-2025`	`final final v2`
`app: country page shows 5 timelines`	`fix`

3. Who does what, and where the code goes

Sandbox part	Owner	Paste into	Writes
Pair session	Together, Yair types	`01_crosswalk.ipynb`	`crosswalk_gw.csv`, `crosswalk_sipri.csv`
A V-Dem	Nicolas	`03_vdem.ipynb`	`vdem.csv`, `vdem_regime_spans.csv`
B UCDP	Nicolas	`02_ucdp.ipynb`	`ucdp_cy.csv`, `ucdp_spans.csv`
A SIPRI	Yair	`04_sipri.ipynb`	`sipri_cy.csv`, `sipri_suppliers.csv`
Pair session	Together	`05_merge.ipynb`	`panel.csv`
C Conflict by regime	Nicolas	`06_analysis.ipynb` Part A	
B Descriptives (arms, war and peace)	Yair	`06_analysis.ipynb` Part A	
Pair session	Together	`06_analysis.ipynb` Part B and C	`event_time.csv`
D Regime and conflict charts	Nicolas	`07_charts.ipynb`, then `app/panels_conflict.py`	
C Arms, democracy and main charts	Yair	`07_charts.ipynb`, then `app/panels_arms.py` and `app/panels_analysis.py`	
App and bonus	Together	`app/streamlit_app.py`	

Order matters: 03 V-Dem comes first, because 01 needs `vdem.csv`. Then 01 Crosswalk, because 02 and 04 need it. 05 Merge needs the clean files from 02, 03 and 04. Part C and D in Nicolas's sandbox and Part B and C in Yair's need `panel.csv`.

Pair sessions

Two fixed sessions a week, about 90 minutes each. One person types, the other reads along and checks row counts. Swap roles halfway. Use the sessions for:

	What
1	The shared notebooks: 01 Crosswalk, 05 Merge, 06 Part B and C, the app
2	Code review: each person explains their main notebook line by line
3	Planning the next days and updating the decisions log in `CONVENTIONS.md`

4. Schedule

Dates	Nicolas	Yair	Together	Done when
until Thu 29 Oct	Merge practice, A1 to A3	Merge practice, A1 to A3	Repo, environments, 00 passes, try_streamlit Ex 1 to 4	Course checkpoint Oct 29
Thu 29 Oct to Thu 12 Nov	03 V-Dem, 02 UCDP	04 SIPRI	01 Crosswalk, 05 Merge, basic app on Streamlit Cloud	`panel.csv` pushed, checkpoint Nov 12
Thu 12 to Thu 26 Nov	Descriptives, regime and conflict charts	Descriptives, arms, democracy and main charts	06 Part B and C, full app, bonus	All pages work, checkpoint Nov 26
Mon 30 Nov			Feature freeze, clean-clone test	Fresh clone runs
Tue 1 to Thu 3 Dec	Slides for your theme	Slides for your theme	Two timed rehearsals (8 min + Q&A)	Checklist ticked
Fri 4 Dec			Presentation, 12:00 to 16:00, KOL-H-309	
until Thu 10 Dec	Your part of the report	Your part of the report	2 to 3 pages on OLAT, written without AI	Report submitted

Ask Justinas or Gustav early whether your conflict onset definition (first conflict year after at least 2 peaceful years) is fine, and which bonus feature counts. Regular commits are part of the grade.

5. When something goes wrong

Situation	What to do
Push is rejected: "updates were rejected"	Your partner pushed first. Pull, then push again. If Git reports a conflict, see the next row.
Merge conflict	Usually only happens if both edited the same file. Keep the version of the file's owner. In GitHub Desktop choose "Use the modified file from main" or "from origin". For a notebook, never edit the conflict markers by hand; keep one version and paste the other person's cells again.
You forgot to pull and already changed files	Commit your changes, then pull, then push (the normal end of a session). Git combines them if you touched different files.
The main notebook is broken after a push	Find the last good commit in History, right click, Revert. Fix it in the sandbox, then paste again.
Checkpoint fails	Read the assert message, it names the problem. Check names against `CONVENTIONS.md`. Print `shape` and `head()` before and after the step.
File over 100 MB	Do not commit it. Add it to `.gitignore`. If already committed, ask before trying to fix history.
Clean CSV from your partner is missing	Pull. If still missing, they have not pushed it yet. Work on the next task that does not need it.
Stuck for more than 30 minutes	Write down the error, your attempt and what you expected. Ask your partner, the tutors, or AI for a hint and an explanation.

Asking AI for help

Paste this, filled in, to get a hint and an explanation:

```
Context: notebook 04_sipri, task A6. Dataframe sipri has columns
iso3, year, supplier, tiv.
Goal: top 3 suppliers per country, the rest as "Other".
Our attempt: [paste code]
Error or problem: [paste error]
Explain what is wrong and give a hint first. Do not write the
solution. We are beginners: suggest simple step-by-step code with
loops and named intermediate results, no compact one-liners.
```

Log every AI use in `AI_LOG.md`, on the same day.

6. Checklists

Before every push

	Check
[ ]	I pulled right before pushing
[ ]	Restart and Run All works on the main notebook
[ ]	All checkpoints print OK
[ ]	I only committed files I own (or a shared file I announced)
[ ]	The commit message says what changed
[ ]	I told my partner what I pushed

Before the presentation

	Check
[ ]	Clean-clone test: fresh clone, new environment, notebooks 00, 03, 01, 02, 04, 05, 06, 07 run, app starts
[ ]	All three app pages and the bonus work for any country
[ ]	README: question, data sources, how to run, who did what, decisions (link to the log in CONVENTIONS.md), AI use (AI_LOG.md)
[ ]	Slides and report merged
[ ]	Two rehearsals with a timer